

Experiment No.	04
Title	Design a Dynamic Webpage using JavaScript (DOM Manipulation)
Course Outcome	CO1

1. Objective / Aim

To design and implement a dynamic webpage using HTML5, CSS3, and JavaScript DOM manipulation — demonstrating a dark/light theme toggle, an image slider, a dynamic to-do list, and a live character counter.

2. System Requirements

Hardware

Processor	Intel Core i5 or equivalent
RAM	8 GB or above
Storage	256 GB SSD / HDD

Software

Operating System	Windows 11 Home
Code Editor	Visual Studio Code with Live Server extension
Browser	Google Chrome / Microsoft Edge
Runtime	None — static HTML file with embedded CSS & JavaScript

3. Architecture / Design

File Structure

folder structure

```
lab4_exp4/  
├── exp4.html  <-- HTML5 + embedded CSS3 + JavaScript (DOM)
```

Page Layout & Functionality

- Theme Switch — toggleTheme() toggles a "dark" class on <body> using classList.toggle().
- Image Slider — Next/Previous buttons cycle through an image array by updating the src via getElementById().
- Quick To-Do List — addTask() reads the input value, creates //<button> elements dynamically with createElement()/appendChild(), and each Remove button deletes its own .
- Character Counter — countChars() runs on the textarea's oninput event and updates a live character count using textContent.

JavaScript DOM Concepts Demonstrated

- classList.toggle() for theme switching
- Array indexing with modulo (%) for circular image navigation
- document.createElement() / appendChild() / remove() for dynamic list items

- Event handling via onclick and oninput attributes
- Live DOM text updates using textContent

4. Source Code — exp4.html

exp4.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title> Dynamic Webpage</title>
<style>
  body {
    font-family: Arial, sans-serif;
    text-align: center;
    background-color: #f3eded;
    color: #222;
    padding: 100px;
  }

  body.dark {
    background-color: #0c0b0b;
    color: #f4f4f4;
  }

  button {
    padding: 10px 18px;
    margin: 9px;
    font-size: 15px;
    border: none;
    border-radius: 6px;
    background-color: #c48282;
    color: white;
    cursor: pointer;
  }

  button:hover {
    background-color: #af5a5a;
  }

  section {
    max-width: 400px;
    margin: 30px auto;
    padding: 20px;
    border: 2px solid #664444;
    border-radius: 20px;
```

```
}

body.dark section {
  border-color: #444;
}

#slideImg {
  width: 100%;
  max-width: 300px;
  border-radius: 8px;
}

input[type="text"] {
  padding: 8px;
  width: 60%;
  font-size: 15px;
  border: 1px solid #ccc;
  border-radius: 8px;
}

ul {
  list-style: none;
  padding: 0;
  text-align: left;
}

li {
  background: #f1f1f1;
  color: #222;
  padding: 8px 12px;
  margin: 6px 0;
  border-radius: 5px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

li button {
  background-color: #999;
  padding: 3px 10px;
  font-size: 12px;
}

textarea {
  width: 90%;
```

```
    height: 80px;
    padding: 8px;
    font-size: 14px;
  }

  #charCount {
    font-size: 13px;
    color: gray;
  }
</style>
</head>
<body>

<h1> Dynamic Webpage</h1>

<!-- 1. Dark mode toggle -->
<section>
  <h2>Theme Switch</h2>
  <button onclick="toggleTheme()">Toggle Dark / Light Mode</button>
</section>

<!-- 2. Image slider -->
<section>
  <h2>Image Slider</h2>
  
  <br>
  <button onclick="prevImage()">Previous</button>
  <button onclick="nextImage()">Next</button>
</section>

<!-- 3. To-do list -->
<section>
  <h2>Quick To-Do List</h2>
  <input type="text" id="taskInput" placeholder="Add a task...">
  <button onclick="addTask()">Add</button>
  <ul id="taskList"></ul>
</section>

<!-- 4. Live character counter -->
<section>
  <h2>Character Counter</h2>
  <textarea id="textArea" placeholder="Start typing..."
oninput="countChars()"></textarea>
  <p id="charCount">0 characters</p>
</section>
```

```
<script>
// 1. Dark mode toggle
function toggleTheme() {
  document.body.classList.toggle("dark");
}

// 2. Image slider
const images = [
  "https://picsum.photos/id/1015/300/200",
  "https://picsum.photos/id/1025/300/200",
  "https://picsum.photos/id/1035/300/200",
  "https://picsum.photos/id/1041/300/200"
];
let currentImage = 0;

function nextImage() {
  currentImage = (currentImage + 1) % images.length;
  document.getElementById("slideImg").src = images[currentImage];
}

function prevImage() {
  currentImage = (currentImage - 1 + images.length) % images.length;
  document.getElementById("slideImg").src = images[currentImage];
}

// 3. To-do list
function addTask() {
  const input = document.getElementById("taskInput");
  const taskText = input.value.trim();
  if (taskText === "") return;

  const li = document.createElement("li");
  const span = document.createElement("span");
  span.textContent = taskText;

  const removeBtn = document.createElement("button");
  removeBtn.textContent = "Remove";
  removeBtn.onclick = function () {
    li.remove();
  };

  li.appendChild(span);
  li.appendChild(removeBtn);
  document.getElementById("taskList").appendChild(li);
}
```

```
        input.value = "";
    }

    // 4. Character counter
    function countChars() {
        const text = document.getElementById("textArea").value;
        document.getElementById("charCount").textContent = text.length + "
characters";
    }
</script>

</body>
</html>
```

5. Compilation / Execution Steps

Step 1 Create a project folder named lab4_exp4/.

Step 2 Create exp4.html inside the folder and paste the source code (CSS and JavaScript are embedded within the file).

Step 3 Open the lab4_exp4/ folder in Visual Studio Code.

Step 4 Install the Live Server extension if not already installed.

Step 5 Right-click exp4.html → select "Open with Live Server".

Step 6 Browser opens at http://127.0.0.1:5500/lab4_exp4/exp4.html.

Step 7 Test each feature — toggle the theme, navigate the image slider, add/remove to-do items, and type in the textarea to verify the live character count.

6. Expected Output / Screenshot

The screenshots below show exp4.html rendered in a Chromium browser after interacting with every feature — three tasks added to the to-do list and sample text typed into the character counter — captured in both light mode and dark mode (after clicking "Toggle Dark / Light Mode").

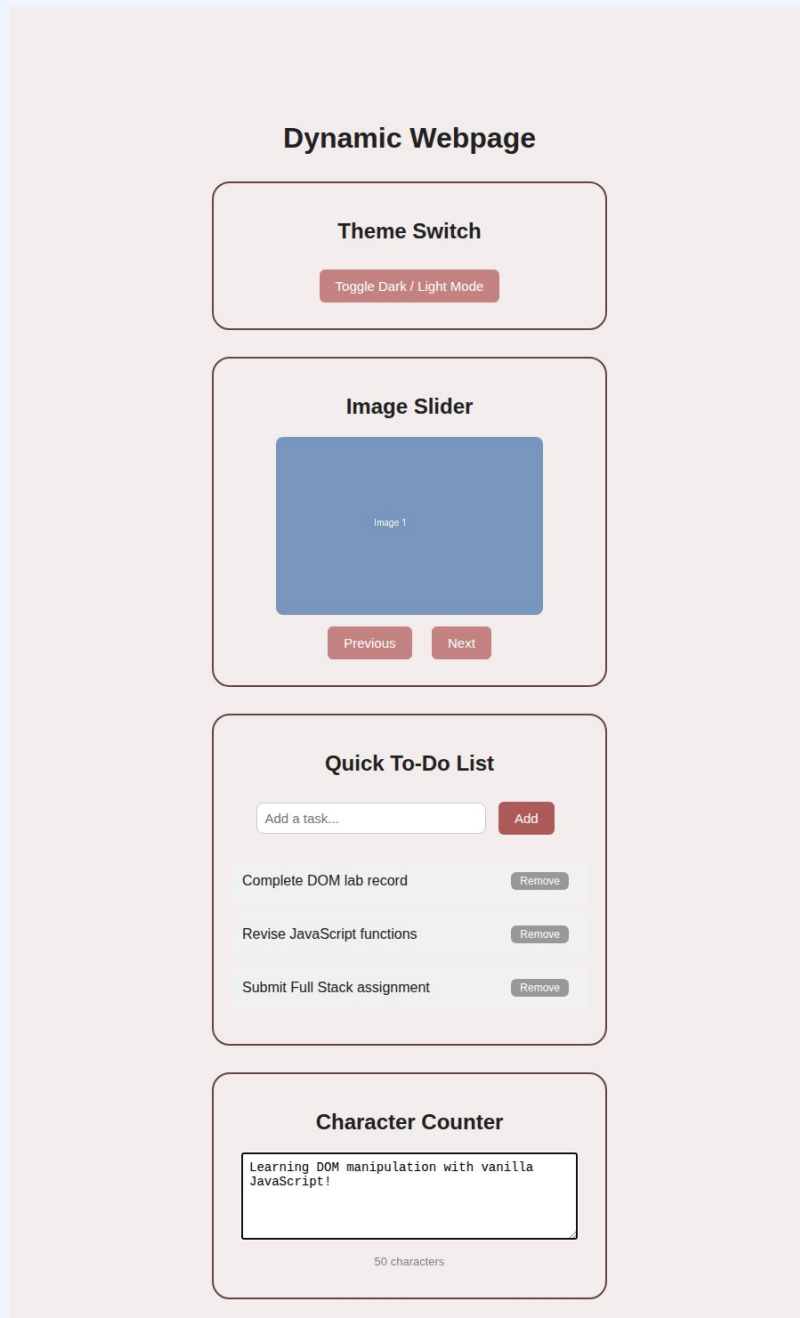


Figure 1: Light mode — to-do list and character counter in action.

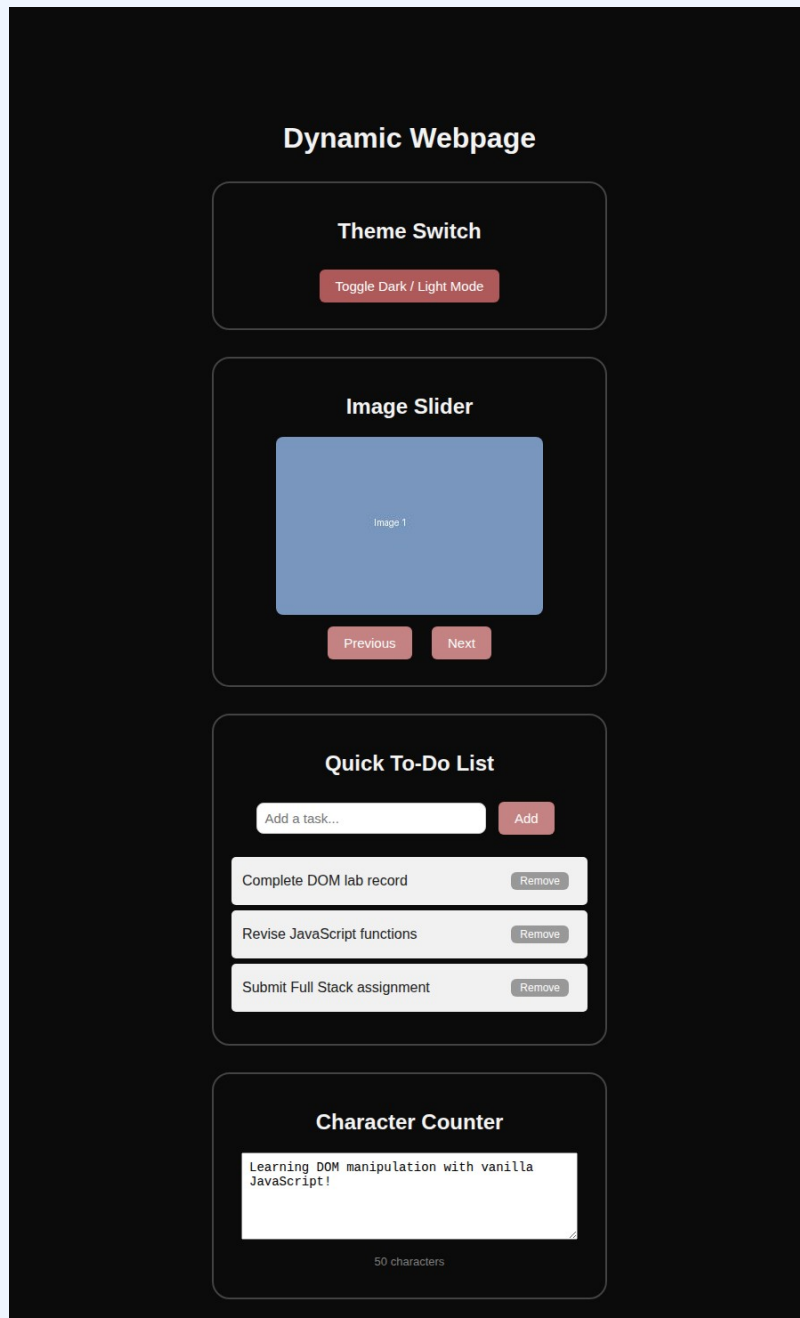


Figure 2: Dark mode — after clicking the theme toggle button.

7. Result / Conclusion

The experiment was successfully completed. A dynamic webpage was designed and implemented using HTML5, CSS3, and JavaScript DOM manipulation, featuring a dark/light theme toggle, an interactive image slider, a fully functional to-do list with add/remove capability, and a live character counter. All four features were verified to update the DOM correctly in real time without reloading the page, demonstrating core JavaScript event handling and DOM manipulation concepts, fulfilling Course Outcome CO1.

Dept of CSE (AIML)

Full Stack Lab — Practical Record

Student Signature

Faculty / Lab In-charge Signature